Akademia Nauk Stosowanych w Nowym Sączu
Wydział Nauk Inżynieryjnych • Informatyka • Studia I stopnia

Niezawodność systemów informatycznych

Zajęcia projektowe nr 5, blok P3

Testy niefunkcjonalne: obciążenie i współbieżność

1 Informacje organizacyjne

- **Blok tematyczny z karty zajęć:** P3: Testy funkcjonalne i niefunkcjonalne
- **Ostatnie zajęcia bloku P3**, po dzisiejszych zajęciach kolejny blok to P4 (analiza statyczna)

Cel dzisiejszych zajęć

Po dzisiejszych zajęciach każdy zespół powinien:

1. rozumieć różnicę między testami funkcjonalnymi a niefunkcjonalnymi,
2. umieć napisać test obciążeniowy mierzący zachowanie systemu pod zwiększonym obciążeniem,
3. umieć napisać test współbieżności wykrywający problemy przy równoczesnym dostępie wielu wątków do tego samego konta,
4. mieć zebrane podstawowe metryki wydajnościowe swojego wariantu SUT.

2 Wprowadzenie teoretyczne

Testy niefunkcjonalne

Testy sprawdzające nie to, *co* system robi, tylko *jak* to robi: jak szybko, jak zachowuje się pod obciążeniem, jak radzi sobie z wieloma równoczesnymi operacjami. W odróżnieniu od testów funkcjonalnych, pojedynczy wynik rzadko wystarcza, zwykle liczy się zachowanie w czasie albo pod rosnącym obciążeniem.

Testy obciążeniowe i wytrzymałościowe

Test obciążeniowy (*load test*) sprawdza zachowanie systemu przy spodziewanym, typowym obciążeniu. Test wytrzymałościowy (*stress test*) zwiększa obciążenie aż do granicy wydolności systemu albo poza nią, żeby sprawdzić, jak system reaguje na przeciążenie, np. czy zwraca czytelny błąd, czy ulega awarii w nieprzewidywalny sposób.

Testowanie współbieżności

Sprawdzanie zachowania systemu, gdy wiele wątków albo procesów wykonuje operacje na tych samych danych jednocześnie. Celem jest wykrycie błędów, które nie ujawniają się przy pojedynczym, sekwencyjnym wywołaniu, np. utraty aktualizacji przy równoczesnym zapisie tej samej wartości przez dwa wątki.

Dlaczego błędy współbieżności są trudne do odtworzenia

Błąd współbieżności zależy od dokładnego momentu, w którym system operacyjny przełączy wątki, a to z definicji jest niedeterministyczne. Ten sam test może przejść dziesięć razy z rzędu i zawieść za jedenastym razem, bez żadnej zmiany w kodzie. Dlatego testy współbieżności uruchamia się wielokrotnie i patrzy na wynik statystycznie, a nie po jednym przebiegu.

Podstawowe metryki wydajnościowe

Przepustowość (throughput): liczba operacji wykonanych w jednostce czasu. **Czas odpowiedzi (latency)**: czas pojedynczej operacji, od wywołania do zwrócenia wyniku. Te dwie metryki nie zawsze rosną i maleją razem, system może mieć wysoką przepustowość i jednocześnie wysoki czas odpowiedzi pojedynczej operacji, jeśli dobrze równoważy wiele operacji naraz.

Szkic testu współbieżności

```
import threading

bank = BankSystem()
bank.utworz_konto("1234567895", "Zrodlo", saldo_poczatkowe=10000)
bank.utworz_konto("9876543215", "Cel", saldo_poczatkowe=0)

def wykonaj_przelew():
    bank.przelew("1234567895", "9876543215", 10)

watki = [threading.Thread(target=wykonaj_przelew) for _ in range(100)]
for w in watki:
    w.start()
for w in watki:
    w.join()

# oczekiwane: saldo konta docelowego wzroslo o dokladnie 100 * 10 = 1000
print(bank.konta["9876543215"].saldo)
```

Jeśli wynik odbiega od oczekiwanej wartości, to sygnał, że pod obciążeniem współbieżnym coś gubi albo duplikuje aktualizacje salda.

3 Zadanie projektowe na dziś

1. **Napiszcie test współbieżności** dla metody modyfikującej saldo konta (np. `przelew` albo `wplac`): wiele wątków wykonujących tę samą operację równocześnie na tym samym koncie, ze sprawdzeniem końcowego salda.
2. **Uruchomcie ten test wielokrotnie** (np. 20 razy) i zanotujcie, czy wynik jest zawsze taki sam.
3. **Napiszcie test obciążeniowy**: wykonajcie dużą liczbę operacji (np. 10 000 wpłat) i zmierzcie czas wykonania.
4. **Porównajcie czas wykonania** tej samej liczby operacji sekwencyjnie i równolegle (wieloma wątkami).
5. **Policzcie przepustowość** (operacje na sekundę) dla obu wariantów.

Jak odróżnić niestabilny test od prawdziwego problemu

Jeśli test współbieżności czasem przechodzi, a czasem nie, to zwykle nie jest błąd samego testu, tylko objaw rzeczywistego problemu w testowanym kodzie. Zanim uznacie test za „niestabilny” i go zignorujecie, uruchomcie go kilkanaście razy i policzcie, jak często zawodzi, to konkretna, policzalna obserwacja do raportu.

Co trafia do raportu

Na koniec dzisiejszych zajęć zespół dopisuje do wspólnego raportu projektowego:

- opis testu współbieżności i wynik jego wielokrotnego uruchomienia (ile razy na ile przeszedł),
- zmierzony czas wykonania dużej liczby operacji sekwencyjnie i równolegle, oraz przepustowość obu wariantów,
- jeśli test współbieżności wykrył niespójność: opis obserwowanego zachowania i warunków, w jakich występuje,
- wnioski: czy testy niefunkcjonalne ujawniły coś, czego nie wykryły testy funkcjonalne z zajęć nr 4.

4 Pytania kontrolne

1. Czym różnią się testy funkcjonalne od niefunkcjonalnych? Podaj przykład pytania, na które odpowiada każdy z tych rodzajów testów.
2. Dlaczego błąd współbieżności może nie ujawnić się za pierwszym uruchomieniem testu, a ujawnić się za drugim, bez żadnej zmiany w kodzie?
3. Czym różni się przepustowość od czasu odpowiedzi pojedynczej operacji?
4. Na czym polega różnica między testem obciążeniowym a testem wytrzymałościowym?

5 Przygotowanie do zajęć nr 6 (blok P4)

Zajęcia nr 6 dotyczą analizy statycznej, czyli oceny jakości kodu bez jego uruchamiania. Na następne zajęcia:

- mieć dostęp do kodu źródłowego własnego wariantu SUT (dziś i na zajęciach nr 2–3 patrzyliście głównie na zachowanie, zajęcia nr 6 wracają do samego kodu),
- zachować wyniki testów z bloków P2 i P3 w repozytorium, posłużą do porównania z wnioskami z analizy statycznej.